

SKAILAMA

Assignment Reference Document

MERN Stack Developer

Event Management System • Multi-Profile & Multi-Timezone

Item	Details
Stack	MongoDB · Express.js · React · Node.js (JavaScript only, no TypeScript)
Effort	Around 10-12 hours
Deadline	Within 1 day of receiving the assignment email
Evaluated on	Code quality, architecture, UI aesthetics, edge cases, originality
Submit	GitHub repo link + Live deployed link + Explanation video

IMPORTANT: DO NOT USE AI TOOLS FOR CODING

This assignment must be written entirely by you, by hand. Genuineness is the whole point of this exercise.

- Not allowed: AI chat assistants that produce code, in-editor AI autocomplete or agents (Copilot, Cursor, Windsurf, Codeium, and similar), and app-generation tools (v0, Bolt, Lovable, and similar).
- Allowed: using AI for reference or guidance only — reading explanations, understanding a concept, checking documentation.

Skailama checks every submission with a tool that measures how much of the code is AI-generated versus human-written. Submissions with AI-written modules above 5-6% will be disqualified.

You will also walk through this code on video and in a live coding round, so you must be able to explain and extend every line. Turn off AI autocomplete before you start, and commit in small steps as you go.

Other rules from the assignment email

- MERN only — no other database or backend language. No TypeScript.
- All data must go through your Express API. Do not store application data in localStorage.
- Your deployment must be fully working when you submit.
- Treat it as a production app: handle edge cases and validate input.
- **Bonus:** prefer vanilla CSS, keep code modular with good use of hooks, and explain the DSA choices you made in your video.

1. Objective

Build a full-stack Event Management System where an admin can create multiple user profiles and schedule events for one or many profiles at once. Every event must display correctly in whichever timezone the viewer selects, and every change to an event must be logged.

The real challenge is not CRUD — it is handling time correctly. An event created at 9:00 AM in India must show as the correct moment for someone viewing in Eastern Time, and stay correct when either user switches timezone.

Reference Video Link :- [Event Management and User Profiles Overview](#)  | [Loom](#)

What you will build

- **Profiles** — admin creates named profiles; a selector at the top decides whose events are shown.
- **Event creation form** — multi-select profiles, pick a timezone, then pick start and end date + time.
- **Timezone-aware event list** — a "View in Timezone" selector that instantly re-renders all timestamps.
- **Edit modal** — loads existing values and saves changes through the API.
- **Update logs (bonus)** — per-event history showing previous versus updated values with converted timestamps.

2. Prerequisites

- JavaScript ES6+, React basics (hooks, props, controlled inputs), Express basics, Mongoose basics, Git.
- CSS layout with flexbox and grid, plus media queries for responsiveness.

Read up on these four things first — **thirty minutes here saves hours of debugging**:

- **UTC vs local time** — why you store one absolute instant and convert it at display time.
- **IANA timezone IDs** — why you use Asia/Kolkata, not +05:30 or IST.
- **Daylight Saving Time** — why New York is -05:00 in January but -04:00 in July, so fixed offsets break.
- **dayjs utc + timezone plugins** — how dayjs.tz(input, zone) reads a wall-clock time in a zone, and how .tz(zone) re-renders an instant elsewhere.

The one idea to remember

An event does not have "a time". It has one absolute instant stored in UTC, plus a timezone label saying where it was created. Every timestamp you show on screen is calculated at render time from that instant and the viewer selected timezone — never stored.

Store the instant. Render the interpretation.

3. Functional Requirements

3.1 Profiles

- Admin can create multiple profiles by entering a name. No delete functionality is needed.
- Names must be trimmed and unique; reject empty or duplicate names with a clear message.
- A new profile must appear in every dropdown immediately, without a page reload.

3.2 Event creation

An event can be created for one or multiple profiles at the same time, with these fields:

Field	Control	Requirement
Profiles	Searchable multi-select with inline "Add Profile"	At least one profile required; multiple allowed.
Timezone	Dropdown of IANA zones	Required. The zone in which the entered times are read.
Start Date & Time	Calendar + time selector	Required. Read in the selected timezone.
End Date & Time	Calendar + time selector	Required. Must be strictly after the start.

- End date/time cannot be before or equal to the start date/time — check on the client for fast feedback and on the server as the real rule.
- Start and end times must respect the selected timezone. The same wall-clock input in two zones gives two different instants.
- After a successful create, clear the form and show the event in the list right away.

3.3 Viewing and updating

- Users can view and update all events assigned to them, filtered by the selected profile.
- Each event stores createdAt and updatedAt, displayed in the viewer selected timezone.
- Each event card shows profile names, start and end date-times, created and updated stamps, plus Edit and View Logs buttons.
- Show a clear "No events found" empty state when a profile has no events.

3.4 Multi-timezone handling

- Each user can pick their own viewing timezone, and all events display in it.
- Changing the viewing timezone must re-render every timestamp instantly — with no data change and no refetch.
- Updates must be correct for every user timezone, which happens automatically if you store absolute instants.

3.5 Bonus: update logs

- Whenever an event is updated, log the change with previous versus updated values and the change timestamp.
- Log timestamps must also convert when the user changes timezone.
- Show "No update history yet" for an event that was never edited. Logs are append-only.

4. Technical Requirements

Layer	Technology	Purpose
Frontend	React 18+ (Next.js also fine)	UI for the form, list, and modals; re-renders when timezone changes.
Backend	Express.js 4+ on Node.js 18+	REST API handling every data operation.
Database	MongoDB (Atlas free tier)	Stores profiles, events, and logs. No other database allowed.

Layer	Technology	Purpose
ODM	Mongoose 8+	Schemas, validation, references, and population.
State	Redux Toolkit, Zustand, or Recoil	Required. Holds profiles, events, current profile, and viewing timezone.
Date/time	dayjs + utc and timezone plugins	Reliable, DST-aware timezone conversion. Preferred by the brief.
Styling	Vanilla CSS or CSS Modules	Vanilla CSS earns bonus points.
Language	JavaScript only	TypeScript is not allowed for this assignment.

Supporting libraries

- **axios** — HTTP calls — wrap it in one API module so the base URL lives in a single place.
- **cors** — lets your deployed frontend call your API.
- **dotenv** — loads MONGODB_URI and PORT from environment variables. Never commit .env.
- **express-validator / zod / joi** — server-side request validation.

Environment

- Node.js 18 LTS or newer with npm 9+.
- A MongoDB Atlas cluster (or local mongod for development).
- Git and a public GitHub repository.
- Hosting: Vercel or Netlify for the client; Render, Railway, or Fly.io for the API.

Avoid these

- Storing dates as display strings like "Oct 15, 2025 09:00 AM".
- Storing fixed offsets like +5.5 instead of an IANA identifier — DST will break them.
- Using localStorage as your data store (saving a UI preference like the last timezone is fine).
- Building extras nobody asked for: authentication, notifications, or recurring events.

5. Application Flow

1. Open the app: header with the current-profile selector, Create Event panel on the left, Events panel on the right.
2. Add a profile from the header dropdown by typing a name and clicking Add.
3. In the form, select one or more profiles, choose a timezone, then pick start and end date + time.
4. Click Create Event. The client converts the wall-clock input to UTC and posts it; the event appears in the list.
5. Change "View in Timezone" — every timestamp re-renders instantly, with no network request.
6. Click Edit. The modal opens pre-filled in the event own timezone, so you see the time originally entered.
7. Save. The API computes what changed, writes log entries, and returns the updated event.
8. Click View Logs to see the change history, timestamped in the current viewing timezone.

How the data moves

```

INPUT    "2025-10-15" + "09:00"    zone = Asia/Kolkata
  |
  v    dayjs.tz("2025-10-15 09:00", "Asia/Kolkata").toISOString()
CLIENT  -> "2025-10-15T03:30:00.000Z"    (absolute instant)
  |
  v    POST /api/events    { profiles, timezone, startAt, endAt }
SERVER  validate (endAt > startAt) -> save -> 201
  |
  v    stored in MongoDB as a UTC Date + the timezone label
RENDER  dayjs.utc(startAt).tz(viewingTimezone).format(...)
        viewing Asia/Kolkata -> Oct 15, 2025  9:00 AM
        viewing America/New_York -> Oct 14, 2025 11:30 PM

```

Both readings are correct — they are the same instant shown in two zones. This is exactly what the reviewer will test.

6. Database Schema Overview

Three collections. Logs live separately so the event document stays small and history stays append-only.

profiles

Field	Type	Notes
name	String	Required, trimmed, unique.
timezone	String	Optional — the profile own preferred viewing zone.
createdAt / updatedAt	Date	Added by Mongoose timestamps, stored in UTC.

events

Field	Type	Notes
profiles	[ObjectId]	References to Profile. Required, at least one.
timezone	String	IANA ID of the zone the event was created in. Required.
startAt	Date	Absolute instant in UTC. Never a formatted string.
endAt	Date	Absolute instant in UTC, must be after startAt.
createdAt / updatedAt	Date	Mongoose timestamps, rendered per viewer zone.

eventlogs

Field	Type	Notes
event	ObjectId	Reference to the Event. Indexed.
field	String	What changed: profiles, timezone, startAt, or endAt.
message	String	Readable summary, e.g. "End date/time updated".
previousValue / updatedValue	Mixed	Raw values (instants, id arrays) — not display strings.
changedAt	Date	UTC instant of the change.

Why this design

- **Reference profiles, do not embed** — renaming a profile should not mean rewriting every event.
- **Keep the timezone with the instant** — you need it to reopen the edit form showing the time originally typed.
- **Separate log collection** — a growing array inside a document eventually hits the 16MB limit.
- **Index profiles and event** — both are always used for filtering, so an index replaces a full scan. Worth mentioning as a DSA point in your video.

7. API Endpoints

Method	Path	Purpose
GET	/api/profiles	List all profiles.
POST	/api/profiles	Create a profile from { name }; rejects empty and duplicate names.
GET	/api/events	List events; ?profileId= filters to one profile. Populates profile names.
GET	/api/events/:id	Fetch one event for the edit modal.
POST	/api/events	Create an event from { profiles, timezone, startAt, endAt }.
PUT	/api/events/:id	Update an event, work out what changed, and write log entries.
GET	/api/events/:id/logs	Fetch that event update history, oldest first.

- Send and receive UTC ISO strings only — never a formatted display string.
- Use a consistent response shape: { success, data } on success and { success, message } on failure.
- Use correct status codes: 200 read/update, 201 create, 400 validation, 404 not found, 500 unexpected.
- Never return a raw Mongoose error to the client — map it to a clean message.

8. Step-by-Step Implementation Guide

Build the backend first and test it with Postman, then build the frontend against a working API. Doing both at once makes bugs much harder to isolate. Rough budget: 1 hour setup, 3 hours backend, 5 hours frontend, 1 hour deploy, 1 hour testing and recording.

STEP 1 Plan before coding

Do: Write down the three collections, the API routes, the component tree, and the store shape. Fifteen minutes in a text file.

Why: Decide up front that instants are stored in UTC and timezone is only for display — otherwise you will rewrite it later.

Outcome: A one-page plan, and a clear answer for "how did you approach this" in your video.

STEP 2 Set up the repo

Do: Create one repo with client/ and server/ folders. Run git init and add a .gitignore for node_modules, .env, and build output.

Why: One repo means one submission link, and committing from the start builds the natural history that shows your own work.

Outcome: A clean repo with an initial commit and no secrets tracked.

```
mkdir skailama-event-management && cd skailama-event-management
git init && mkdir client server
printf "node_modules\n.env\ndist\nbuild\n" > .gitignore
```

STEP 3 Scaffold Express and connect MongoDB

Do: In server/, install express, mongoose, cors, dotenv, and nodemon. Connect to MONGODB_URI first, and only then start listening.

Why: Connecting before listening makes a bad connection string fail loudly at boot instead of as a confusing 500 later.

Outcome: npm run dev prints a success message, and GET /api/health returns 200.

STEP 4 Define the Mongoose models

Do: Create Profile, Event, and EventLog as in Section 6. Enable timestamps, mark required fields, and add a validator so profiles has at least one entry.

Why: Schema validation is your safety net — even if a controller bug lets bad data through, the database refuses it.

Outcome: Invalid documents are rejected with clear Mongoose validation errors.

STEP 5 Build the profile routes

Do: Add GET and POST /api/profiles. Trim the name, reject empty strings, and turn the duplicate-key error (code 11000) into a 400.

Why: Profiles are the simplest resource, so use them to settle your layering, response shape, and error handling before the harder parts.

Outcome: You can list and create profiles in Postman, and a duplicate name returns a readable 400.

STEP 6 Build event creation

Do: Add POST /api/events. Check that profiles is a non-empty array of real ids, the timezone is a valid IANA id, both instants parse, and endAt is strictly after startAt. Save, then populate profile names.

Why: This is the core of the assignment. Converting on the client and validating instants on the server keeps the ordering rule timezone-independent.

Outcome: A valid payload returns 201; an end-before-start payload returns 400 and writes nothing.

STEP 7 Build listing and single fetch

Do: Add GET /api/events with an optional profileId filter using { profiles: profileId }, sorted by startAt and populated. Add GET /api/events/:id returning 404 when missing.

Why: This is your most frequent query, so add an index on profiles — a good, concrete DSA point for your video.

Outcome: Filtered and unfiltered lists both work, and a bad id returns 404 instead of crashing.

STEP 8 Build update with diff logging

Do: Add PUT /api/events/:id. Load the existing document, re-validate the merged values, save, then compare before and after and insert one log per changed field.

Why: Computing the diff yourself means the history is always truthful, and saving with no changes writes no misleading entries.

Outcome: Changing only the end time creates exactly one log entry; saving identical values creates none.

STEP 9 Add the logs route and error handling

Do: Add GET /api/events/:id/logs sorted oldest first, returning raw UTC values. Then add a 404 handler and one central error middleware, and wrap async controllers so rejections reach it.

Why: If the server pre-formats log timestamps, they can never re-convert when the viewer changes timezone. And one error boundary makes every failure predictable instead of crashing the process.

Outcome: Logs return in order, and a deliberately broken request returns clean JSON while the server stays up.

STEP 10 Scaffold React and the timezone utility

Do: Create the client with Vite and install your state library, dayjs, and axios. Register the utc and timezone plugins in one setup file. Then write utils/datetime.js with three functions: toUtcInstant, formatInZone, and toZonedFormFields.

Why: Registering plugins once prevents the "dayjs.tz is not a function" error, and centralising conversion means there is only one file to check when a timestamp looks wrong.

Outcome: No component calls dayjs.tz directly — everything goes through your utility.

STEP 11 Build the state store

Do: Create the store with profiles, currentProfileId, viewingTimezone, events, logs, and ui state, plus the actions to fetch and mutate each. setViewingTimezone must only set state.

Why: The header, form, list, and both modals all share this state — exactly why an external library is required. Deriving the filtered list with a selector avoids stale-UI bugs.

Outcome: Changing the viewing timezone re-renders everything with zero network activity in the browser network tab.

STEP 12 Build the dropdowns

Do: Write one reusable dropdown that closes on outside click and Escape. Use it twice: single-select with inline add in the header, and multi-select with search and tick marks in the form.

Why: These controls differ only in selection mode, so extracting one component is the modularity the brief asks for — and the reviewer can see it.

Outcome: Both work by mouse and keyboard, a new profile shows up in both instantly, and the form reads "2 profiles selected".

STEP 13 Build the create form

Do: Assemble profiles, timezone, and the two date + time rows. Validate on change and disable submit while invalid or in flight. On submit, convert to UTC through your utility and post.

Why: Comparing start and end inside the selected timezone (not browser-local time) is what makes the rule correct.

Outcome: Invalid input is blocked with inline messages; a valid submit creates the event and clears the form.

STEP 14 Build the list and timezone selector

Do: Render each event as a card and format every timestamp with formatInZone using viewingTimezone from the store.

Why: Because formatting is derived at render, one selector change updates the whole list correctly — no stored strings to go stale.

Outcome: Switching timezone visibly shifts every timestamp, and the empty state appears when a profile has no events.

STEP 15 Build the edit modal

Do: Pre-fill the modal with `toZonedFormFields(instant, event.timezone)`, reuse the same validation, then convert to UTC and PUT.

Why: Pre-filling in the viewing timezone instead of the event timezone silently changes the event time on the next save — reviewers look for this exact bug.

Outcome: Opening the editor from a different zone still shows the original time, and saving unchanged values writes no logs.

STEP 16 Build the logs modal (bonus)

Do: Fetch the logs and render each entry with its message, previous and updated values, and the change time via `formatInZone`. Show "No update history yet" when empty.

Why: Formatting log times through the same helper makes the "logs convert with timezone" requirement work for free.

Outcome: Logs are chronological, and changing timezone while the modal is open converts every timestamp inside it.

STEP 17 Style it and make it responsive

Do: Write vanilla CSS with variables for colours, spacing, and radii. Build the two-column layout, cards, dropdown panels, and modal, then stack the columns below about 900px.

Why: UI aesthetics are explicitly part of the grade, and vanilla CSS earns bonus points.

Outcome: The app looks deliberate at both desktop and mobile widths, with no overflow or overlap.

STEP 18 Test, deploy, and record

Do: Work through Section 12, fix what fails, deploy both halves with production environment variables, verify the live URL in a private window, then record your walkthrough.

Why: The brief requires a fully functional deployment — a broken link undermines otherwise good work.

Outcome: A live URL a stranger can use end to end, plus a video covering architecture, timezone strategy, and DSA choices.

9. Project Structure

```

skailama-event-management/
├── client/
│   └── src/
│       ├── api/
│       │   ├── axiosClient.js    # all HTTP calls live here, nowhere else
│       │   ├── profileApi.js     # base URL + shared error handling
│       │   └── eventApi.js
│       ├── components/
│       │   ├── common/           # Dropdown, Modal, Button, Spinner
│       │   ├── profiles/         # CurrentProfileSelect, ProfileMultiSelect
│       │   └── events/           # CreateEventForm, Calendar, EventList,
│       │                           # EventCard, EditEventModal, EventLogsModal
│       ├── hooks/                # useOutsideClick, useEventForm, useDebounce
│       ├── store/                # useProfileStore, useEventStore
│       ├── utils/
│       │   ├── dayjsSetup.js     # register utc + timezone plugins ONCE
│       │   ├── datetime.js       # all conversion + formatting helpers
│       │   └── timezones.js      # friendly label <-> IANA id map
│       ├── styles/               # variables.css, global.css
│       └── App.jsx
├── server/
│   └── src/
│       ├── config/db.js          # mongoose connection
│       ├── models/               # Profile.js, Event.js, EventLog.js
│       ├── controllers/          # request/response only, no db queries
│       ├── services/             # business logic + diff computation
│       ├── routes/               # profileRoutes.js, eventRoutes.js
│       ├── middleware/           # validateRequest, notFound, errorHandler
│       ├── utils/                # ApiError, asyncHandler, timezone check
│       ├── app.js                # express app + middleware + routes
│       └── server.js              # connect db, then listen
├── .gitignore
└── README.md                     # setup, env vars, links, decisions

```

What each part is for

- **client/src/api** — the only code that knows about HTTP, so the base URL changes in one file.
- **components/common** — presentation-only pieces that know nothing about events — this is where reuse shows.
- **hooks** — extracted behaviour; useOutsideClick alone is reused by three dropdowns.
- **store** — shared state. Selectors derive the filtered list; nothing formatted is kept here.
- **utils/datetime.js** — the single source of truth for time. If a timestamp is wrong, this is the only file to open.
- **controllers vs services** — controllers stay thin (parse, call, respond); services hold the logic and are testable alone.
- **middleware/errorHandler.js** — the one place any failure becomes a response.
- **README.md** — the reviewer reads this first — setup, env vars, live and video links, and your key decisions.

10. Validation and Error Handling

10.1 Rules

Rule	Enforced where	On failure
Profile name required, trimmed, unique	Client + server + schema	400 with a readable message, never a 500.
At least one profile per event	Client + server + schema	Submit disabled; API returns 400.
Timezone required and a valid IANA id	Client + server	400 for an unrecognised zone.
Start and end date + time all required	Client + server	Submit disabled; API returns 400.
End strictly after start, compared in the event zone	Client + server + pre-validate hook	Inline message; 400; nothing saved.
Event id must be a valid ObjectId	Server	400 if malformed, 404 if not found.
Times cross the wire as UTC ISO only	Server contract	Reject unparseable values with 400.

10.2 Error handling

- Server: one custom error class with a `statusCode`, thrown from services and caught by a single error middleware registered last.
- Server: wrap async controllers so a rejected promise cannot escape and crash the process.
- Server: map Mongoose `ValidationError`, duplicate key 11000, and `CastError` to clean 400s. Log the real error, send a safe message.
- Client: normalise errors in your axios layer so components always get a plain message string.
- Client: store the error and show it inline — a `console.error` is not error handling.
- Client: keep the user input when a submit fails, and reset the submitting flag in a finally block.

Instead of showing	Show
Request failed with status code 400	End date/time must be after start date/time.
E11000 duplicate key error	A profile with that name already exists.
Cast to ObjectId failed	That event could not be found.

11. Evaluation Criteria

Area	Weight	What the reviewer looks for
Functionality	25%	Profiles, multi-profile events, viewing, updating, and the end-after-start rule all work.
Timezone correctness	20%	UTC storage, correct display conversion, correct edit pre-fill, IANA ids, instant re-render.

Area	Weight	What the reviewer looks for
Code quality	20%	Clear layering, small reusable modules and hooks, no duplication, no dead code or stray logs.
UI / UX	15%	Clean layout close to the snapshots, consistent spacing, responsive, all states handled.
Edge cases & errors	10%	Server-side validation, right status codes, readable messages, graceful failure.
Update logs (bonus)	5%	Accurate history with previous vs updated values, converting with timezone.
Video explanation	5%	Clear walkthrough of architecture, timezone strategy, and DSA choices.

Extra credit

- Vanilla CSS with design tokens instead of a component library.
- Real reuse: one dropdown used three ways, one form hook shared by create and edit.
- DSA reasoning explained on video — a Map for O(1) name lookups, a Set for comparing profile arrays, an index instead of a collection scan, a debounce on search.
- A README a reviewer can follow to run the project in five minutes, plus a clean commit history.

Avoid at all costs

- **AI-generated code above the 5-6% threshold — immediate disqualification.**
- TypeScript, another database, or storing data in localStorage instead of MongoDB.
- A broken deployment, a missing link, a committed .env file, or a missed deadline.

12. Bonus Challenges

Finish every core requirement first. A polished core submission always beats a half-finished ambitious one.

- **Event update logs** — the named bonus in the brief, and the highest-value addition.
- **Per-profile timezone** — store a timezone on each profile so switching profile also switches the viewing timezone.
- **Overlap warning** — flag when a new event overlaps an existing one for any selected profile — sort by start and do one linear pass.
- **Search and filter** — debounced text search plus a date-range filter on the event list.
- **Duration display** — show a readable event duration, correct across a DST boundary.
- **Accessibility pass** — aria roles on custom dropdowns, focus trapping in modals, visible focus styles.
- **A few unit tests** — for your conversion helpers and diff function — the highest-value things to test.

Do not add authentication, calendar integrations, notifications, or recurring events. None are asked for, and each one adds bugs the reviewer will find.

13. Submission Guidelines

All three items are required. A missing link counts as an incomplete submission.

What to submit

1. GitHub Repository Link

- A public repo with the complete source for both client and server.
- A README covering setup, environment variables, the live link, the video link, and your key decisions.
- No .env file, credentials, or node_modules committed anywhere.

2. Published / Deployed Project Link

- A live, public URL where the whole flow works, with the frontend pointing at the deployed API.
- Verify it in a private window on another device before submitting.

3. Screen Recording Link

- A short video explaining the project, its features, and a code walkthrough.
- Make sure the link is publicly viewable without a permission request.

Where and when

- Deadline: within 3 days of receiving the assignment email.
- Submit on the official Google Form: [assignment submission form](#)
- Reply to the assignment email once you have submitted, and confirm your intention to proceed before you start.

How to structure your video

1. Demo (2-3 min): create a profile, create a multi-profile event, switch timezone and show the timestamps convert, edit it, show the logs, resize the window.
2. Architecture (2-3 min): folder structure, the routes-controllers-services layering, schema design, and your store.
3. Timezone strategy (2 min): the most important part — UTC storage, conversion at render, why IANA ids matter, and why the edit form pre-fills in the event zone.
4. DSA and optimisation (1-2 min): explicitly requested — name the data structures and complexity improvements you chose and why.
5. Edge cases and trade-offs (1 min): what you validated, and what you chose not to build.

Build it carefully, build it yourself, and be ready to explain it.

Assignment walkthrough recording: <https://www.loom.com/share/5528d6e35a184007a37e222949400279>